BYTES / IDENTITY / CONSUMERS / AUTHORITY

File Upload Vulnerabilities Explained

Follow the file, prove the weakness, and secure the full upload lifecycle

EXACT BYTES

What artifact was checked?

NEXT CONSUMER

What may it do?

Understand the file before testing its boundaries.

Find the weakness. Prove the effect. Verify the repair.

Practice with real bytes and bounded local workflows.

Haroon Shahid

First edition / Version 1.0 / 1 November 2026

Before you begin

File Upload Vulnerabilities Explained

Follow the file, prove the weakness, and secure the full upload lifecycle

By Haroon Shahid. First edition, version 1.0. 1 November 2026. Prepared with AI assistance.

Copyright 2026 Haroon Shahid. The purchased publication is licensed for personal use under LICENSE.txt. The separately identified lab code is MIT licensed. The free preview may be shared unchanged with attribution. No resale or public redistribution of the paid publication is granted.

This book teaches responsible analysis through fictional examples and bounded local practice. It does not certify expertise. Use only systems and accounts you own or are explicitly authorized to assess.

The validation report identifies executed tests and limits. Framework, cloud, converter, and isolation guidance is version-sensitive. The local DocDock lab uses real multipart, bytes, a restricted PNG profile, bounded ZIP extraction, local capabilities and manual jobs. Its scanner is a dummy marker policy. It is not a production security library.

PDF code can soft-wrap for page fit. The offline HTML and source preserve original lines. Responsive HTML and scalable diagrams provide an alternative reading format. On narrow screens, swipe sideways within wide diagrams and tables. No full PDF accessibility conformance is claimed.

Free preview: introduction, roadmap and Chapters 1-3. The full package includes all 56 chapters, answered reviews, the offline reading edition, practice workbooks, reference sheets, and the local DocDock lab.

Contents

Before you begin 2
Contents 3
Introduction 4
Learning roadmap 6
01. Follow a file beyond the Upload button . . . 8
02. Files are bytes; names are claims . . . 10
03. Understand the request and the response . . . 12
Sources and further reading 14

Introduction

A picture is accepted. What has been proved?

Mira adds a photograph to DocDock. A green badge says success. Somewhere behind that badge, bytes were received, labels were interpreted, a storage identity was chosen and perhaps a worker was queued. Later a browser or another program will read the result. Each step can preserve the feature's promise or quietly change it.

This book teaches you to follow that journey. You will learn what a file is before testing file types, what a path means before testing traversal, what a browser origin grants before testing active documents, and what a version means before testing approval and queued work. The advanced ideas stay connected to the first simple question: what is this file allowed to do next?

Learn a method, not a collection of suffixes

File upload security spans content, location, ownership, work, interpretation and audience. Some failures need executable syntax. Others need only a harmless text file and two accounts. A valid image can be private. A rejected upload can leave public bytes. A clean scan can approve the wrong replacement. A safe preview can coexist with an unsafe original.

The useful skill is predicting these transitions and testing them precisely. You will identify a feature contract, map consumers, preserve exact artifact identity, choose a bounded paired proof, explain the observed consequence, repair the failed grant and verify normal use. You will also learn to state what remains unknown.

A connected story with real local practice

DocDock is fictional. Mira and Ben belong to different fictional companies. Their photographs, invoices and markers contain no real customer data. The lab handles real multipart requests and bytes, writes actual files, parses a narrow PNG profile, extracts bounded ZIP members and checks local signed links.

Weak and safe paths make the important choices visible. A fixed SVG event demonstrates browser document behavior. A playpen demonstrates an unintended write location while retaining outer containment. A dummy scanner makes verdict identity easy to inspect. A manual queue makes changed-source consumption reproducible without distributed infrastructure.

The scanner is not antivirus. The PNG parser is not a production image library. The lab never executes uploaded server code, fetches arbitrary external URLs, touches arbitrary host paths or uses cloud credentials. Its privileged audit is explanatory training visibility. Those boundaries make the examples repeatable; they are also limits on what the results prove.

How to use the ecosystem

Each chapter has a goal, an analogy, an original teaching diagram, a worked explanation, three knowledge checks and a claim/check/consumer/proof revision view. Answer explanations explain why, rather than only naming a correct choice. The companion practice deck adds predictions, guided missions and worked review cases. Reference sheets, templates and the mastery tracker support later use.

Read a small connected group. Close the book and explain the idea. Predict before revealing an answer. Perform one local experiment. Write a fact, an inference and a limit. Revisit the explanation the next day, after a week and after a month. If you are confused, return to the

introducing chapter linked in the glossary.

What completion means

A book cannot guarantee an expert title, a vulnerability discovery, income or permanent coverage of every future parser. The valuable outcome is that you can independently investigate an unfamiliar upload feature and produce evidence another person can review. Use only systems and accounts you own or are explicitly authorized to assess. Agree on recipients, resource limits and effects before testing delayed consumers.

The source catalog distinguishes consulted sections from further reading. The validation report distinguishes executed tests, deterministic model checks, documented architecture patterns and untested platforms. That honesty is part of the teaching method: precision about evidence is itself an advanced security skill.

Learning roadmap

Reader	Route	Produce this evidence
New learner	Read in order; use bundled samples and glossary	A file passport and correct evidence-ladder explanations
Developer	Foundations, relevant processors, Chapters 33-44 and 50	A feature contract and meaningful allow/deny tests
Authorized tester	Foundations, Chapters 9-32 and 49-56	Paired proof, consumer map and bounded report
Architect	All lifecycle stages and capstone	Identity, worker, audience, change and revocation design

Seven stages with visible gates

1. **Understand files and trust, 1-8.** Bytes, metadata, HTTP, multipart, paths, formats, identity and evidence. Gate: explain why accepted, valid, private and executable are different claims.

2. **Find and prove, 9-16.** Map surfaces, build paired probes, compare labels with bytes, resolve ambiguity and complete the first local repair. Gate: reproduce a bounded finding with a normal baseline.

3. **Understand accepted content, 17-24.** Locations, replacement, archives, work budgets, competing interpretations, browser/server consumers and quarantine. Gate: identify the precise boundary and required impact prerequisites.

4. **Follow the lifecycle, 25-32.** Images, documents, XML resources, scanners, jobs, storage capabilities, direct finalization and caches. Gate: keep source, verdict, output and delivery identity connected.

5. **Build defenses, 33-40.** Product contracts, bounded receiving, normalization, names, workers, delivery, actions and framework responsibilities. Gate: preserve normal use while denying the failed grant.

6. **Review modern workflows, 41-48.** APIs, resumable transport, imports, tenants, change, chains, advisories, AI readers and full architecture. Gate: label deployment assumptions and produce a complete acceptance matrix.

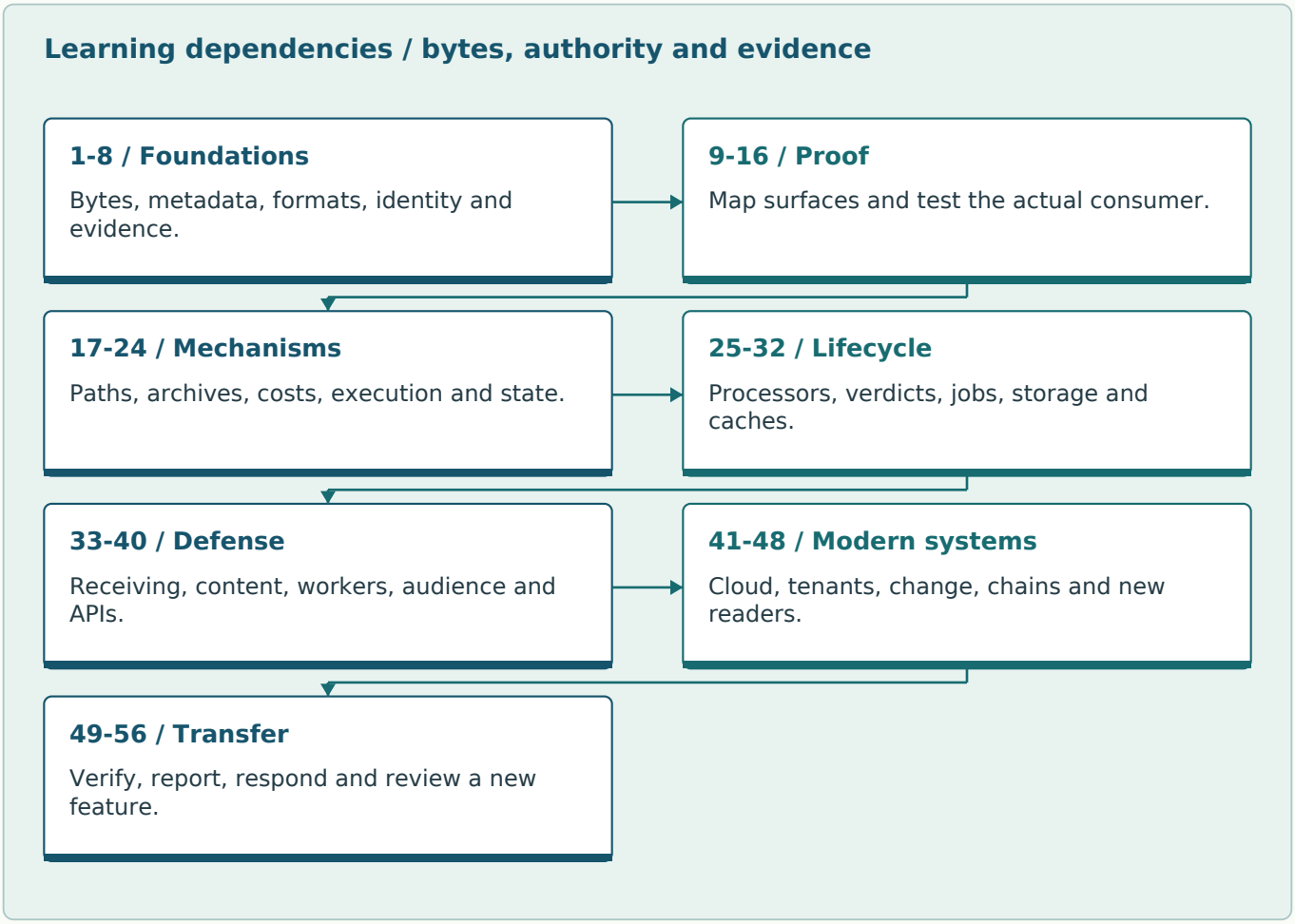
7. **Verify and transfer, 49-56.** Manual review, regression, false conclusions, reporting, logging, response and independent capstone. Gate: a new review another person can reproduce without your verbal explanation.

A flexible study plan

Take one or two chapters per session. Do not rush past an unfamiliar prerequisite to reach a more dramatic exploit. After each part, explain the map from memory, solve several prediction cards and perform a relevant mission. The 40 predictions train decisions; 32 missions connect concepts with local execution; 24 worked synthetic cases train transfer and partial-fix judgment.

Use the mastery tracker as an evidence record, not a certificate. Mark a skill demonstrated when you can explain it, perform a bounded test, interpret the result and name its limits. Review the weak points after a day, a week and a month. Advanced readers can move faster through familiar foundations while still checking the gates.

The reference flow models are compact reminders. They do not replace chapters, standards or deployment integration tests. Cloud, converter, framework, operating-system isolation and distributed-cache behavior must be verified in the real environment.



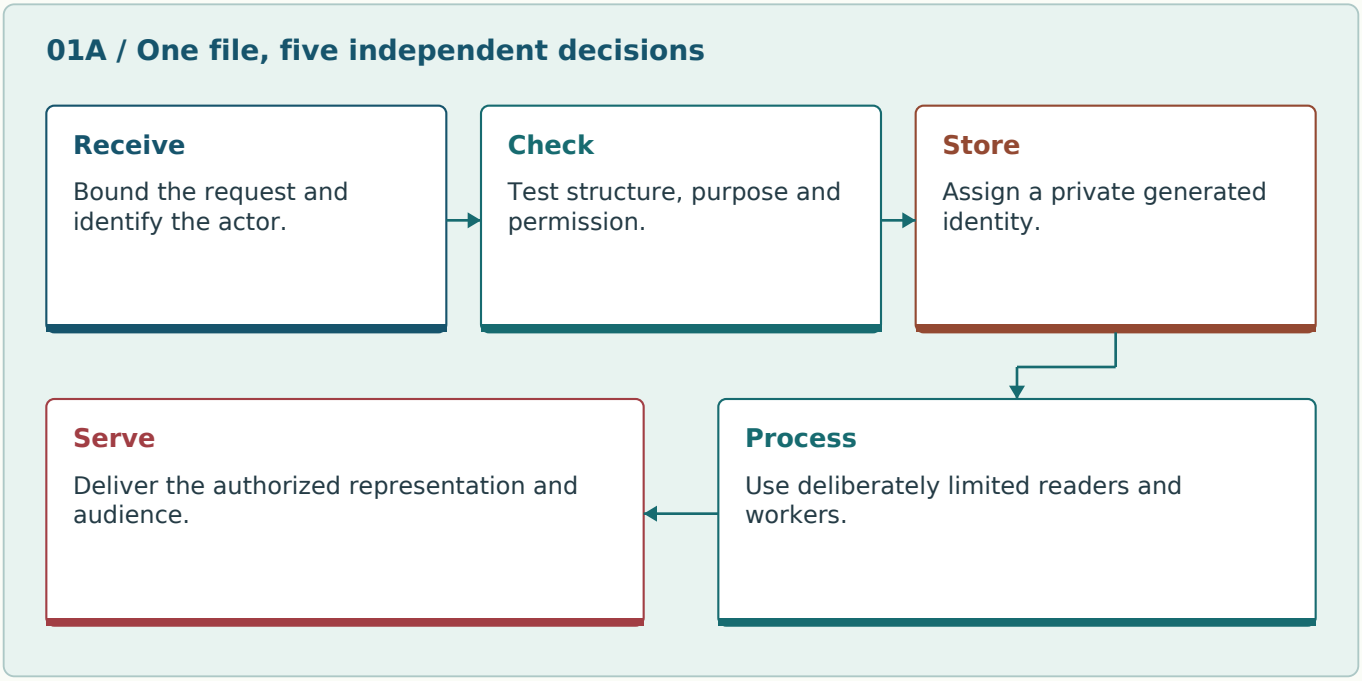
Read left to right within each stage, then follow the connector to the next row. Each later stage uses the earlier mental model.

I. UNDERSTAND FILES AND TRUST

01. Follow a file beyond the Upload button

Learn to: Describe receiving, checking, storing, processing and serving as separate decisions.

Think of it this way: A parcel can pass the front desk and still be unsuitable for a kitchen, a public display or a locked office.



Follow the file after acceptance; the next consumer often determines the consequence.

Meet DocDock

Mira uses DocDock, a fictional document service. She adds a profile picture and a private invoice. Ben has a different account in a different company. The picture should become a small public image. The invoice should remain available only to its permitted readers. Both begin with the same Upload button, but their promises differ.

An **upload** transfers bytes from a client to a receiving service. Bytes are small units of stored information. A **file upload vulnerability** exists when an upload workflow lets someone cross a security boundary: read another person's document, place bytes in an unintended location, execute unwanted instructions, publish active content with excessive authority, or consume resources beyond an allowed budget.

Acceptance alone is not that boundary. A service may intentionally store many file types without interpreting them. It may reject a harmless format because the product supports only photographs. Security analysis begins with what the feature promises and what each component actually does.

Five verbs worth remembering

Receive the request within a budget. **Check** the intended format and permissions. **Store** the object under an application-controlled identity. **Process** it using deliberately limited capabilities. **Serve** the right representation to the right audience. Later chapters add versions, jobs, sharing and deletion, but these five verbs stay useful.

A successful response can mean only that a queue accepted work. A preview might be a newly generated image rather than the original. A private download might pass through a CDN, a cache near the reader. Ask which bytes and which decision produced the result.

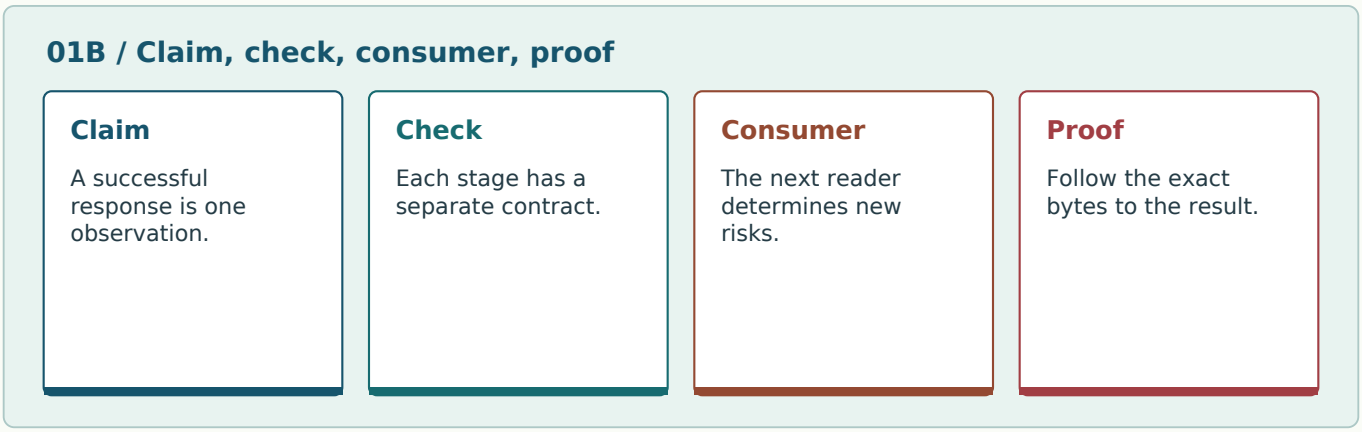
Your first prediction

Mira uploads a text file named portrait.png. The service stores it privately and always returns it as an attachment to Mira. Does that prove code execution? No. It establishes acceptance and storage of unexpected bytes. To claim more, locate a consumer that gives those bytes additional meaning or authority.

Now suppose the same bytes replace Ben's invoice because both files use the original filename as their storage key. The demonstrated failure is an unauthorized replacement, even though nothing executes.

Start a file passport

Write the owner, intended audience, original name, stored identity, version, checks, consumers and retrieval route. A **consumer** is any component that reads or interprets the file. This passport becomes your investigation map. Keep its claims small enough that you can prove each one.



Key takeaway. Follow the file after acceptance; the next consumer often determines the consequence.

Check your understanding

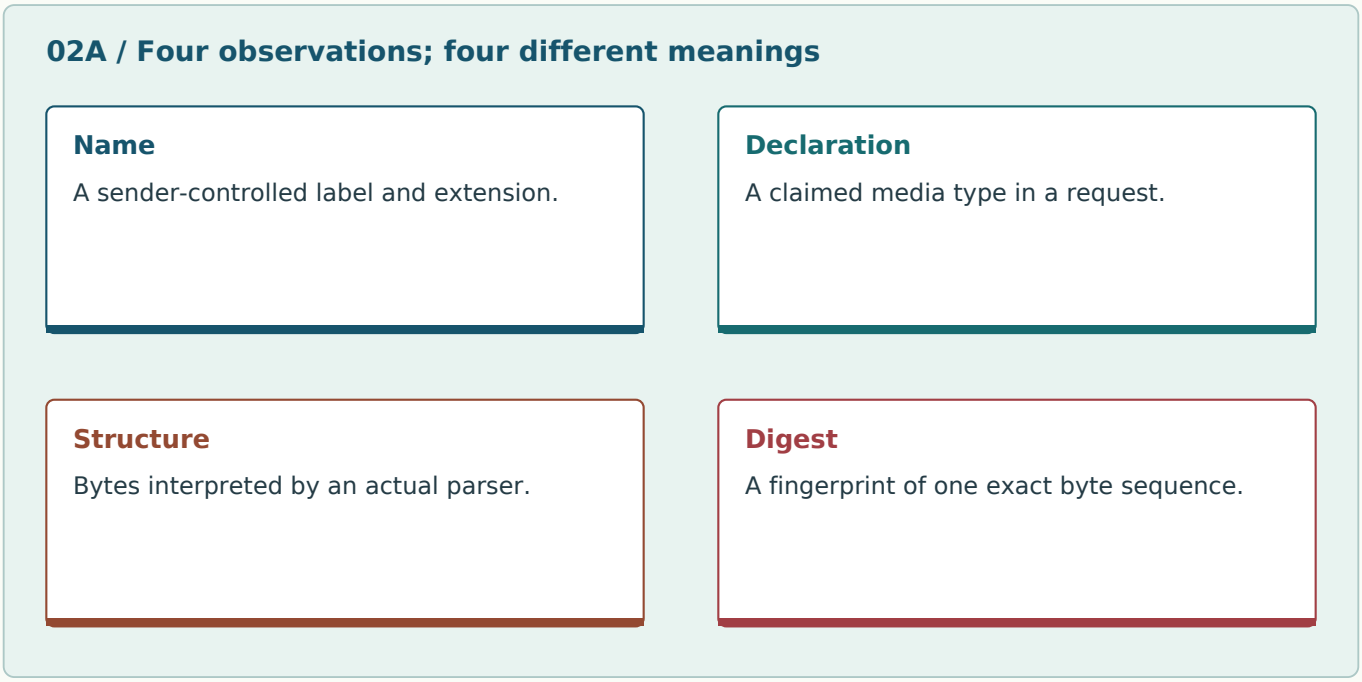
1.1 Does acceptance prove execution?	**1.2** Why can two uploads need different policies?	**1.3** What is a file passport?

Answer explanations are in the full ebook. Sources: OWASP, ACADEMY, AUTH

02. Files are bytes; names are claims

Learn to: Separate filename, extension, length, digest and format.

Think of it this way: A label saying glass does not turn a cardboard box into glass. Weighing the box also does not reveal its contents.



A name, a media type, a valid format and a safe use are different claims.

What a file contains

A file is a sequence of bytes. A byte can represent a number from 0 to 255. Text is one way to interpret bytes. Images, archives and documents use other rules. The same bytes can have different meanings to different readers.

A **filename** is a label such as portrait.png. An **extension** is a name ending, here .png. Changing the ending does not rewrite the bytes. A file's **length** is its byte count. Length helps enforce budgets but does not identify a format or a safe reader.

A **digest** is a compact value calculated from bytes using a hash function. This book uses SHA-256. It is useful for checking whether a recorded verdict refers to the same content. It is not a malware verdict, an ownership check or encryption. A person who can replace both the file and an unprotected digest record can replace the evidence together.

Read three independent facts

Suppose the request says filename=portrait.png, the part says Content-Type: image/png, and the body starts with plain text. The name and type are claims from the sender. The body is the content the next parser will read. A server should not silently treat those three things as equivalent.

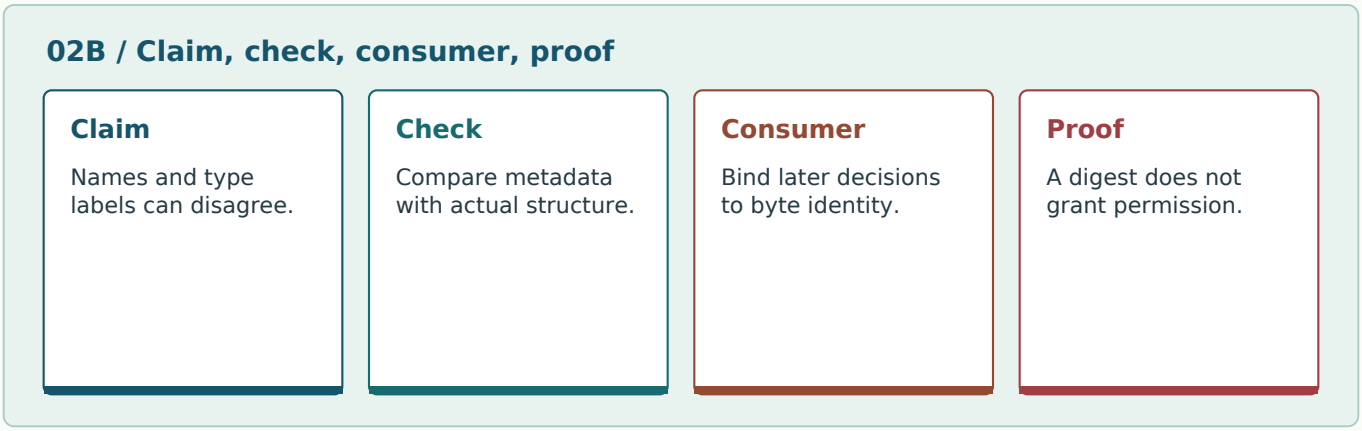
A **signature**, sometimes called magic bytes, is a familiar byte pattern near the start of a format. PNG begins with eight particular bytes. A text file with that prefix is still not necessarily a valid PNG. Structure, lengths, checksums, compressed data and completion matter too.

Identity through change

Mira's first upload has digest A. A replacement has digest B. The object ID may remain unchanged while the bytes change. An antivirus result for A cannot approve B merely because both belong to attachment 42. Versions let you name this change explicitly. An immutable version is a version whose bytes do not change after creation.

Try writing a table with four columns: name, declared type, observed structure and digest. Put an ordinary PNG in the first row. Put text renamed to .png in the second. Put the same PNG under a different name in the third. Predict which columns change.

Later you will bind scan results, background jobs and signed links to byte identity. For now remember the difference: a label describes a claim; a digest describes content identity; a parser checks a format contract. None of them decides who may read the object.



Key takeaway. A name, a media type, a valid format and a safe use are different claims.

Check your understanding

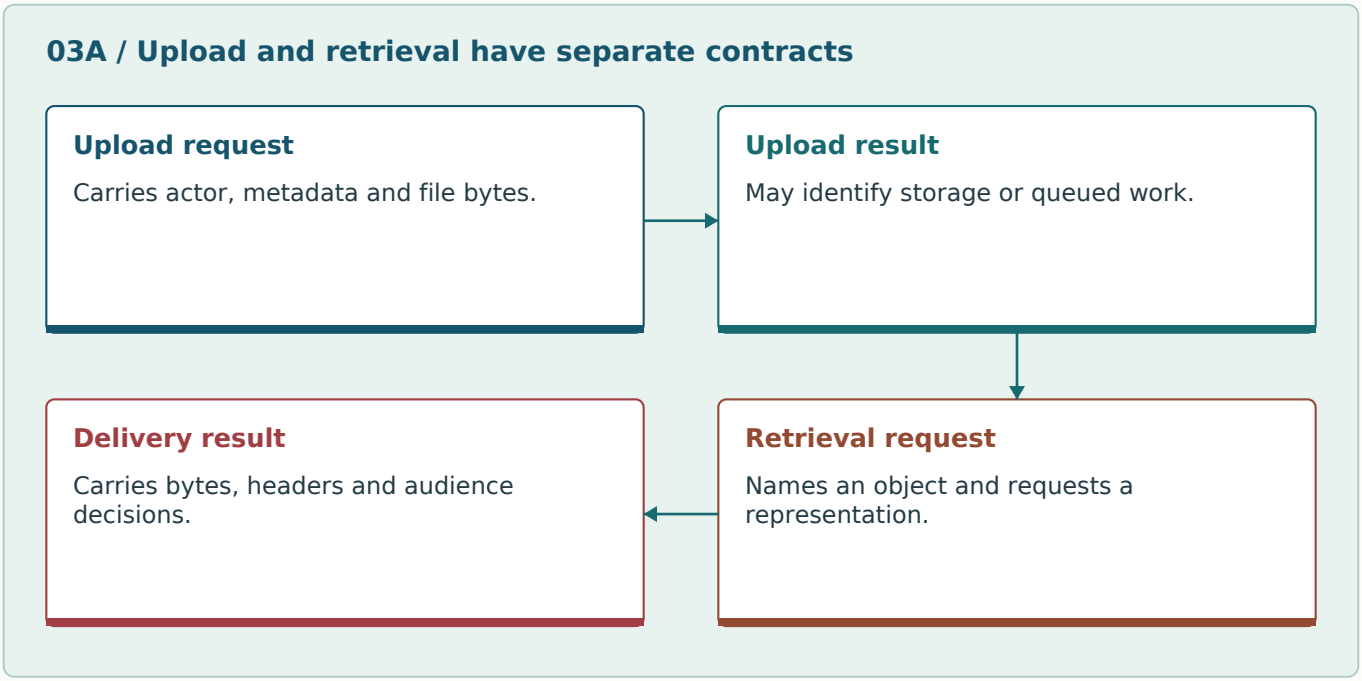
2.1 Does renaming change file content?	**2.2** What does SHA-256 help establish?	**2.3** Can a valid PNG be unsafe to publish?

Answer explanations are in the full ebook. Sources: PNG, HASH, OWASP

03. Understand the request and the response

Learn to: Read the parts of HTTP and separate uploading from retrieving.

Think of it this way: Sending a parcel to a warehouse and asking the warehouse to deliver it are different transactions.



Read both upload and retrieval; a safe decision on one route does not prove the other.

Two programs exchange messages

A client starts an interaction. A server handles it. A browser is one client; a mobile application or command-line tool can be another. HTTP is the message protocol they use. HTTPS adds transport protection through TLS. It does not decide whether a file should be published or executed.

A **URL** is a resource address. In http://127.0.0.1:8950/file/abc, http is the scheme, 127.0.0.1 is the host, 8950 is the port selecting a listener, and /file/abc is the path. A query after a question mark carries additional parameters. A domain name is a human-readable host name; DNS resolves names to network addresses. Local loopback addresses refer to your own computer.

A request has a method, target, headers and sometimes a body. POST commonly submits an upload. PUT can send object bytes to a chosen resource. A GET commonly retrieves content. These names do not enforce the application's permissions by themselves.

```
POST /attachments HTTP/1.1
Host: app.example.test
Content-Type: multipart/form-data; boundary=DemoBoundary
Content-Length: 247
```

The length above is illustrative, not the size of a complete reusable request. A real client calculates the length from its actual body. The boundary describes how to separate body parts. We open that envelope in the next chapter.

Responses are evidence, not verdicts

A response includes a status, headers and often a body. A 201 may describe a newly created object. A 202 may describe accepted queued work. An application can use statuses incorrectly, so also read its result and inspect stored state in the authorized workflow.

The **upload response** might be JSON containing an attachment ID. The **download response** later contains the bytes. They can have different authentication, cache, media type and disposition rules. A safe upload check cannot repair a public retrieval endpoint that forgets ownership.

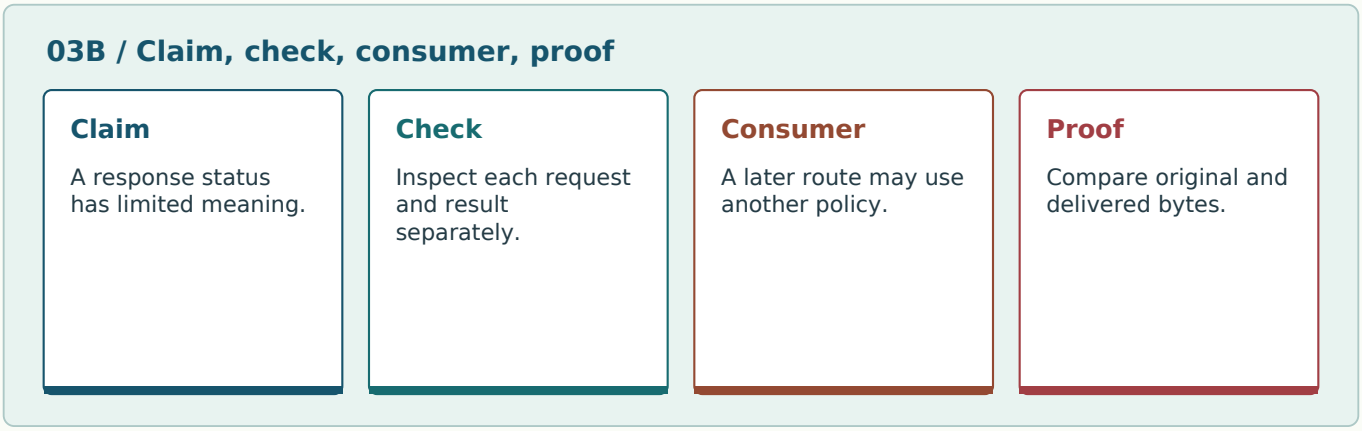
Where the bytes go

A reverse proxy is a service in front of the application. It may buffer a request, impose a size limit, forward it, or reject it. The application may then write to temporary disk or cloud storage. A worker may read it later. Every layer can have a separate budget and timeout.

Look in the browser's Network panel for the request URL, method, fields, response and later preview request. Do not infer that the preview is the original file. Compare lengths and digests when you have access to both.

A useful distinction

Downloading an uploaded script as literal text is different from a server running it. Seeing an image is different from a document executing script. Seeing a login page instead of bytes may mean retrieval was denied, not that upload failed. Keep request, storage and consumer observations in separate notes. This prevents the most common exaggerated reports before testing becomes complicated.



Key takeaway. Read both upload and retrieval; a safe decision on one route does not prove the other.

Check your understanding

3.1 Does HTTPS make uploaded files safe?	**3.2** Why inspect a preview request?	**3.3** Does literal script text prove server execution?

Answer explanations are in the full ebook. Sources: HTTP, MULTIPART, ACADEMY

Sources and further reading

Research cutoff: 1 November 2026. The catalog distinguishes consulted material from targeted further reading. It does not claim every linked page was audited end to end or every client was deployed. Live documents change. Historical fixes are not current release recommendations. Original diagrams and synthetic exercises do not imply vendor endorsement.

OWASP - OWASP: File Upload Cheat Sheet

Consulted targeted sections. Layered upload controls and limits; a design checklist rather than a universal safe-file test.

https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

ACADEMY - PortSwigger: File upload vulnerabilities

Consulted targeted sections. Primary educational research on acceptance, retrieval, interpretation and controlled testing.

https://portswigger.net/web-security/file-upload

HTTP - RFC 9110: HTTP Semantics

Targeted further reading. Request and response meaning, methods, representations and content metadata.

https://www.rfc-editor.org/rfc/rfc9110.html

MULTIPART - RFC 7578: multipart/form-data

Consulted targeted sections. Part boundaries, disposition, filenames, repeated fields and encoding rules. The filename* parameter is not permitted here.

https://www.rfc-editor.org/rfc/rfc7578.html

PNG - W3C: PNG Specification, Third Edition

Consulted targeted sections. PNG chunks, checksums, image structure and decoding. DocDock intentionally implements only a small teaching profile.

https://www.w3.org/TR/png-3/

HASH - Python: hashlib

Targeted further reading. SHA-256 digest operations and byte identity.

https://docs.python.org/3.12/library/hashlib.html

AUTH - OWASP: Authorization Cheat Sheet

Targeted further reading. Object, operation and tenant permissions through the full lifecycle.

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html